Tier-Aware Compressed GPU Vector for CLIO / IOWarp

GPU-native error-bounded compression on the Gray-Scott workload — Delta A100 · cuSZp · CLIO embedded runtime

STATUS — WORKING. The compressed GPU vector round-trips write→evict→compress(HBM)→store→decompress→device-read, and stores a dataset 2× the GPU-memory budget entirely on the GPU (16× capacity). Verified on A100.

What this is

A **vector-like abstraction whose pages live across HBM → DRAM → NVMe → PFS** with automatic **GPU** compression. Data produced by a GPU kernel (here, a Gray-Scott reaction-diffusion simulation) is stored compressed **without ever leaving the GPU to compress**: pages are compressed in-place in HBM by cuSZp (an error-bounded lossy float codec) and only the small compressed result is written down the tier stack through CLIO's real runtime. The compressor is a CLIO chimod placed in front of the CTE core; the Vector<T> routes its page evictions and faults through it transparently.

Compressed GPU vector — end-to-end result

A Vector<float> (256 KiB pages) is written on-device; the async cache manager **evicts every dirty page through the compressor**, which compresses it in HBM and stores the compressed blob in the CTE core. Reading back, FaultAllSync() decompresses every page straight into its HBM slot, then a device read kernel reads them. Verified on A100 (cuSZp, abs error bound 1e-3):

Stage	Pages	Per page	Result
Evict → compress (HBM)	4 / 4	262144 B → ~16.6 KB	~16:1
Store (compressed, in core)	4 / 4	compressed blob	ok
FaultAllSync → decompress	4 / 4	→ HBM slot	ok
Device read kernel	262144 elems	max_abs_err 1.0e-3	== eb

max_abs_err = 1.0e-3 (equals cuSZp's error bound), mean 4.9e-4 — the loss is exactly the requested bound, i.e. correct. Test: cte_gpu_vector_compress_cuda (PASS). No regression in the existing cte_gpu_vector_cuda.

How it is wired (no DRAM copy to compress)

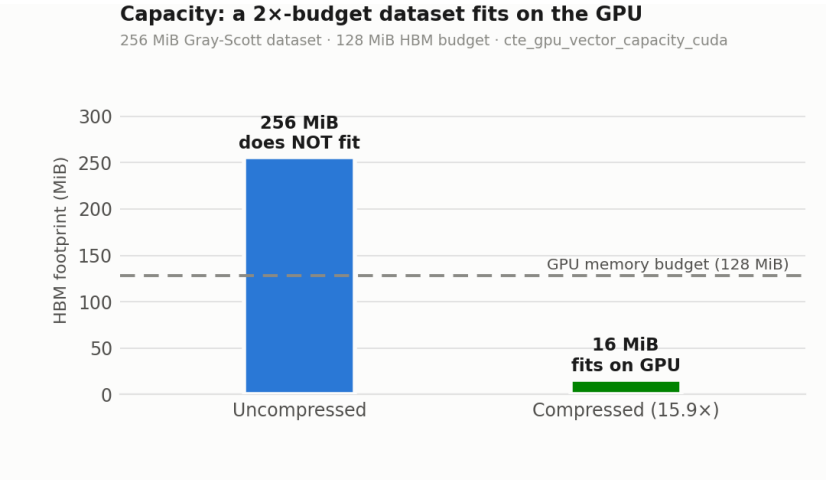
Zero-copy HBM	Page data is passed as a device pointer; cuSZp compresses it in place using its own temp HBM buffer. Nothing is staged to DRAM to compress — only the compressed bytes leave the device.
Compressor handlers	Raw core PutBlob / GetBlob arriving at the compressor entrypoint are transparently compressed / decompressed and forwarded to the core (per-page name <tag>_b<blk>_pi<page>).
Vector routing	New storage_pool_id ctor arg: page traffic goes to the compressor pool; the tag stays on the core, which the compressor forwards to. A plain vector becomes compressed with one argument.
Library pin	CLIO_CTE_COMPRESS_LIB=cuszp selects the GPU codec; the factory (WireIdForName/MakeCuszp) registers it. lz4/zstd (CPU) also available.

Capacity win — a dataset larger than GPU memory, stored on the GPU

The headline benefit: because compression shrinks cold pages, a dataset **larger than the GPU-memory budget fits entirely on the GPU**. The compressor's storage tier is a **kHbm bdev (device memory)**, so compressed cold pages live in HBM, not host DRAM. A Gray-Scott field of logical size **2× the HBM budget** was streamed through

the compressed vector; measured HBM footprint (kHbm-tier used bytes):

Quantity	Value	Fits on GPU?
Logical dataset	256 MiB (2× budget)	—
HBM budget	128 MiB	—
HBM used (compressed 15.9×)	16 MiB	yes — 111 MiB to spare
Uncompressed would need	256 MiB	no — exceeds budget

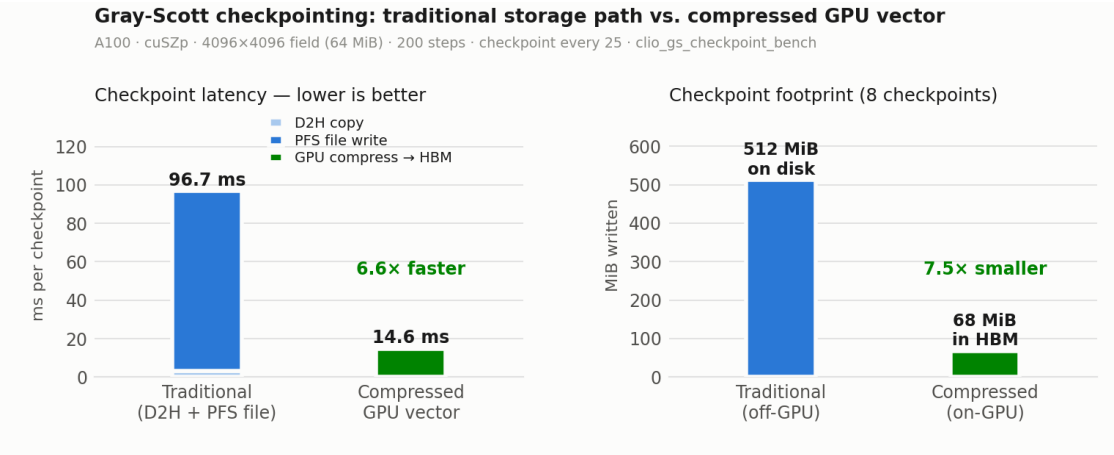


So the same GPU holds ~16× more logical data. Chunk-0 read-back (via FaultAllSync) matched within the error bound (max_abs_err 1.0e-3). Test: cte_gpu_vector_capacity_cuda (PASS). Streaming is chunked + host-paced because paging larger-than-HBM *on-device* deadlocks under GPU compression (see limitation below).

Head-to-head vs. the traditional checkpoint path

Same canonical Gray-Scott loop (iterate steps, checkpoint the field every N steps) checkpointed two ways and timed on the same evolving snapshots: the **traditional path** (cudaMemcpy device→host, then write the full uncompressed field to a Lustre/PFS file — what an HDF5 checkpoint does) vs. the **compressed GPU vector** (compress in HBM, blob stays on the GPU). A100, 4096×4096 field (64 MiB), 200 steps, checkpoint every 25:

Checkpoint path	Latency	Eff. BW	Footprint	Where
Traditional (D2H + PFS file)	96.7 ms	0.65 GiB/s	512 MiB	disk
Compressed GPU vector	14.6 ms	4.27 GiB/s	68 MiB	HBM
Advantage	6.6× faster	6.6×	7.5× smaller	on-GPU



The traditional path is dominated by the PFS write (single-stream Lustre ~0.68 GiB/s); the compressed path is compute-bound (cuSZp) but never leaves the GPU and writes 7.5× fewer bytes. Bench: clio_gs_checkpoint_bench.

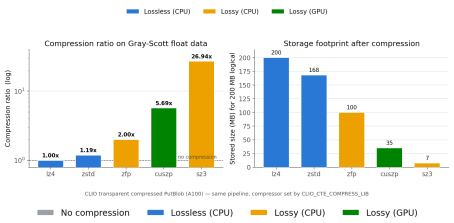
What we reused vs. changed

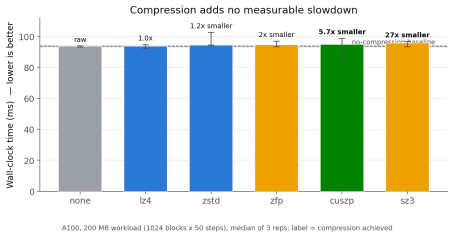
The Vector<T> API was **already implemented** — we reused it and made two small, backward-compatible **additions**; nothing was rewritten.

Reused as-is	The whole Vector<T>: ctor, Device(), FlushAllSync(), the device-side write_range/read_range, the HBM/DRAM paging + cache-manager machinery, legacy/async modes, cold-miss fault, and the existing passing test (left untouched).
Added #1 — one ctor arg	storage_pool_id (defaults to the core = old behavior). Pointed at the compressor pool, it routes page evictions/faults through compression — this is what turns a plain vector into a compressed one.
Added #2 — one method	FaultAllSync() (+ a small resident-marking kernel) to materialize compressed pages back into HBM for device-side reads.
Outside the vector	Compressor PutBlob/GetBlob handlers, task serialization cases, and a new test file — not modifications to the Vector.

Transfer micro-benchmark (context)

Before the vector, a 4-case Gray-Scott transfer benchmark characterized the codec on the same data (raw / async / compressed / async+compressed). Directly-measured compression ratio and the compress-vs-raw slowdown:





Device-side reads: the fault deadlock, and the fix

The transparent on-ACCESS device fault is undeadlockable on one GPU (kernel co-scheduling). The fix is the #700 Transaction API: host-orchestrated WINDOWED PREFETCH.

An on-device page fault spin-waits on the GPU for its GetBlob, but the compressor services that fault by launching cuSZp's decompress kernel on the **same** GPU — and a spin-waiting kernel and the decompress kernel do not co-schedule, so they deadlock. Two fixes were tried and both failed: (1) a dedicated non-blocking stream + cudaMallocAsync in the wrapper and a cuSZp source patch; (2) a preallocated pinned staging pool + pre-warmed mempool. Both remove every device-synchronizing CUDA call, yet the fault still hangs — proving the residual cause is kernel **co-scheduling**, not device-sync. (Uncompressed faults are fine: serviced by a CPU memcpy, no GPU kernel. Eviction is fine too: flush kernels exit before the compressor runs.)

The working answer is the issue-#700 Transaction API — host-orchestrated **windowed prefetch**. Pages are pulled into HBM *ahead* of use while the GPU is idle, so the device kernel only ever reads resident pages. Transaction<T> + SequentialTransaction / PseudoRandomTransaction (with Vector::PrefetchWindowSync) sweep a dataset far larger than the HBM cache one window at a time. Verified A100: an 8 MiB dataset swept in 1

MiB windows (8× the resident footprint) through both a Sequential and a PseudoRandom transaction — 2,097,152 elems each, max_abs_err 1.0e-3 == eb, no on-device fault. Remaining perf work: pipeline the next window's prefetch on a copy stream while the kernel reads the current one.

Environment: A100-SXM4-40GB · CUDA 12.6 (iowarp/deps-nvidia, Apptainer) · cuSZp GPU codec · CLIO embedded runtime (compressor pool → CTE core → HBM/RAM bdev). Key commits: c069d50 (transparent compress), c59eb5c (compressed vector), ce1cc9a (FaultAllSync), bcda63a (capacity), 6077f9f (checkpoint bench), 89c4735 (Transaction API).